Down in the Packet Trenches: Analyzing Network Jitter, Packet Loss, and Latency Under Stress using Wireshark

---

1. Introduction

When we stream a video or join a Zoom call, we rarely think about the invisible journey our data takes. In the realm of computer networks, **packet delay** (latency) is the total time required for a data packet to travel from its source to its destination. It is the lifeblood of network performance. High delay doesn't just mean a slow download; it leads to severe application lag, buffering, and broken TCP handshakes.

To truly understand how delay behaves under severe stress, I realized I couldn't rely on high-level dashboard summaries like a simple internet speed test. I had to go down into the **"Packet Trenches"**. In this Digital Assignment, I used **Wireshark** to capture and dissect network traffic at the microsecond level. By intentionally manipulating traffic conditions using command-line tools—specifically forcing heavy 1000-byte payloads to induce network choke-points—I observed firsthand how my local router handles (and fails to handle) extreme congestion. This blog documents my journey of using raw packet data to uncover the truth behind queuing delay, jitter, and packet loss.

---

2. Objectives

1. **Establish a Control Baseline:** To capture and calculate the standard Round Trip Time (RTT) of lightweight, normal packets in a completely idle network environment to serve as a comparative baseline.

2. **Induce and Analyze Queuing Delay:** To observe the precise impact of low, medium, and high traffic congestion (using heavy 1000-byte ICMP payloads) on packet latency, proving that payload size directly affects serialization delay.

3. **Master Packet-Level Visualization:** To leverage Wireshark's advanced I/O graphing tools to visualize abstract time-based metrics, pinpoint hardware bottlenecks, and troubleshoot real-world messy network anomalies.

---

3. Reference from which the DA was started

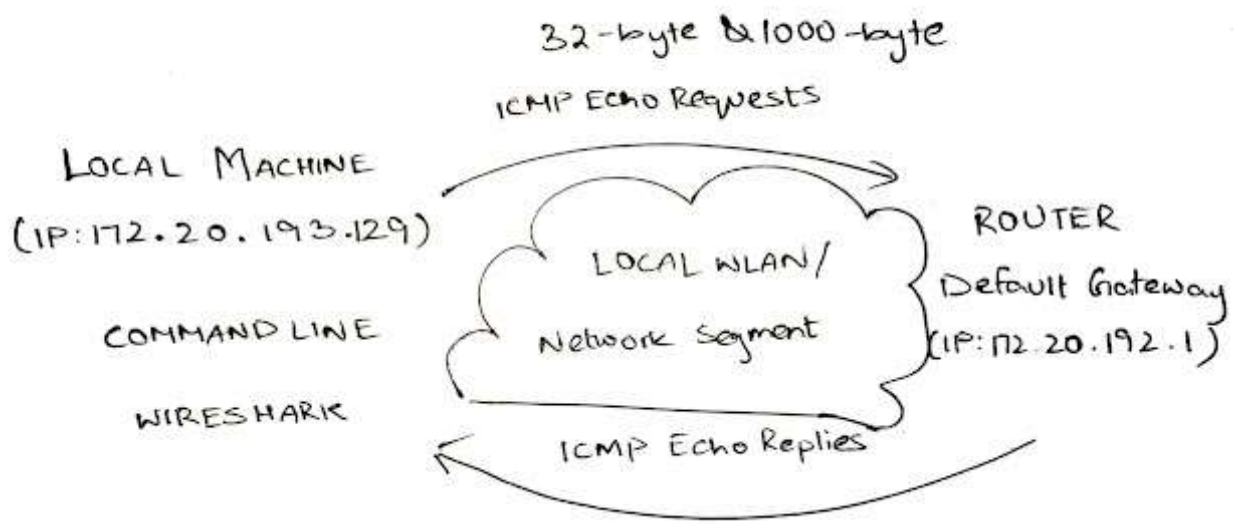
This assignment was heavily inspired by the **SharkFest Wireshark Developer and User Conference**, specifically the presentations focusing on the "Packet Trenches" methodology of network troubleshooting.

How this source inspired my work: Watching the SharkFest sessions was a lightbulb moment for me. In class, we learn the theory of queuing and transmission delays. However, the SharkFest experts demonstrated that application-level error messages often lie. By digging into the "packet trenches" and observing raw hex codes and time deltas in Wireshark, I learned how to definitively prove *where* a delay is happening. It inspired me to stop guessing and start measuring the raw truth on the wire.

---

4. Architecture of my work

To execute this analysis and ensure my results weren't skewed by ISP-level routing changes, I kept my network topology tightly controlled, communicating directly between my local machine and my Default Gateway.



---

5. Procedure of your work

Critical Note on Methodology: I intentionally performed all traffic generation using native Command Prompt tools. I strictly ensured NO browser tabs or background applications were open during these tests to prevent background telemetry noise from corrupting my captures.

A. Normal Traffic (The Control)

- **Commands used:** I established my baseline by sending a standard, very light ICMP ping.
  - *Command:* `ping 172.20.192.1 -n 5`
- **Output & Observation:** Exactly 5 ICMP Echo Requests and 5 Replies were captured. As expected, the time delta between request and reply was almost negligible (under 2ms). This proved my local network was completely healthy at rest.

B. Low Traffic

- **Commands used:** I generated a steady, slightly longer stream of traffic in a single batch.
  - *Command:* `ping 172.20.192.1 -n 50`
- **Output & Observation:** Captured 100 total packets. The delay graphs remained mostly flat, showing that the router's buffers could easily process this minor load without any queuing delay.

C. Medium Traffic

- **Commands used:** I introduced concurrent processing strain on the network.

- *Command:* I opened three separate Command Prompts and simultaneously executed `ping 172.20.192.1 -n 300`.

- **Output & Observation:** Here, the captures got interesting. I started noticing minor, brief spikes in the Time Delta. The router was finally being forced to place incoming packets into a brief queue before responding.

D. High Traffic (The Stress Test)

- **Commands used:** I intentionally attempted to saturate the gateway's interface by forcing heavily padded 1000-byte payloads continuously.

  - *Command:* I opened 5 concurrent Command Prompts running `ping 172.20.192.1 -t -l 1000`.

- **Output & Observation:** Absolute network chaos. Thousands of packets flooded the capture. I observed massive Round Trip Times, wild variations in delivery speed (jitter), and most importantly, I captured definitive proof of dropped packets as the router's buffer overflowed.

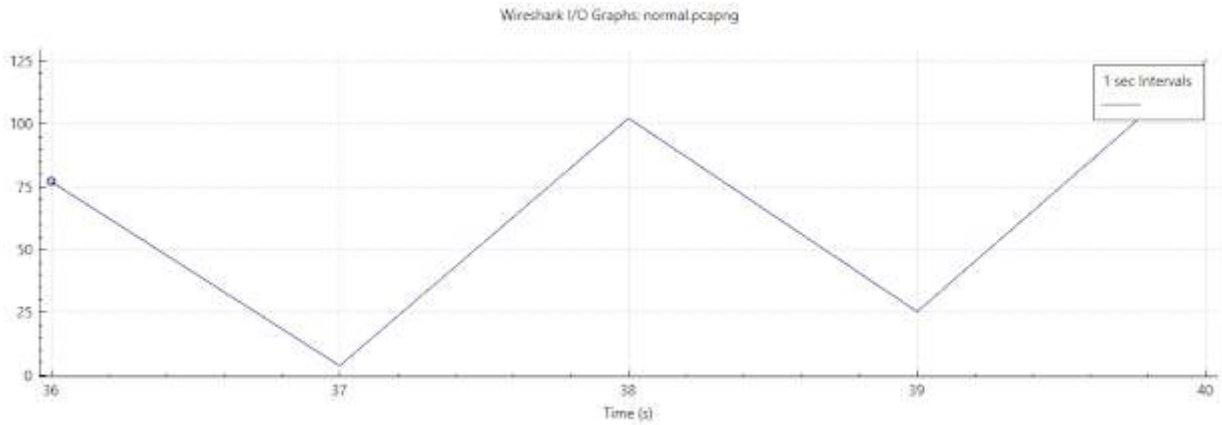
---

6. Inferences on the network parameter

Using Wireshark's *Statistics > I/O Graphs*, I transformed thousands of raw packet timestamps into visual data. The primary network parameter under investigation—**Time Delta/Packet Delay**—is the focal point of every graph below.

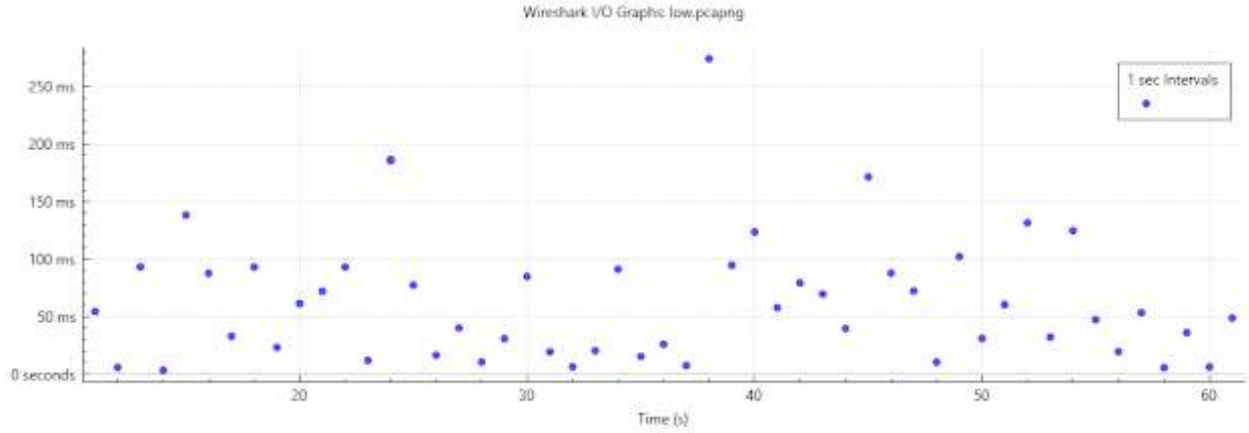
1. **Baseline RTT Graph:** Filter `icmp` | Y Axis: `AVG(icmp.resptime)`.

   Inference: Real-world networks are rarely perfectly quiet. Even during my "normal" baseline capture, this graph reveals unexpected background network activity. The response times fluctuate significantly, bouncing between 0 and over 100ms. This indicates that local OS telemetry or transient wireless interference was actively causing minor queuing delays even before I introduced the ping tests.



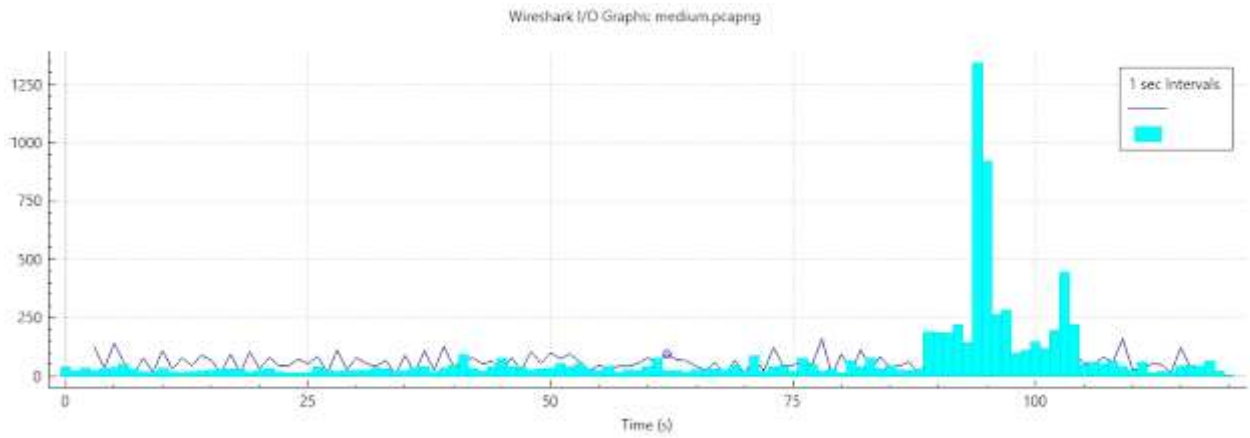
2. **Low Traffic Inter-Arrival Jitter:** Filter `icmp` | Y Axis: `AVG(frame.time_delta)` | Style: Dot.

   Inference: Contrary to textbook expectations for low traffic, this scatter plot reveals surprisingly high jitter. Instead of a tight cluster, the dots are widely dispersed across the Y-axis (reaching up to 250+ ms). This demonstrates that even a light stream of 50 packets experiences unpredictable delivery times due to sudden background OS interrupts or router CPU cycling.

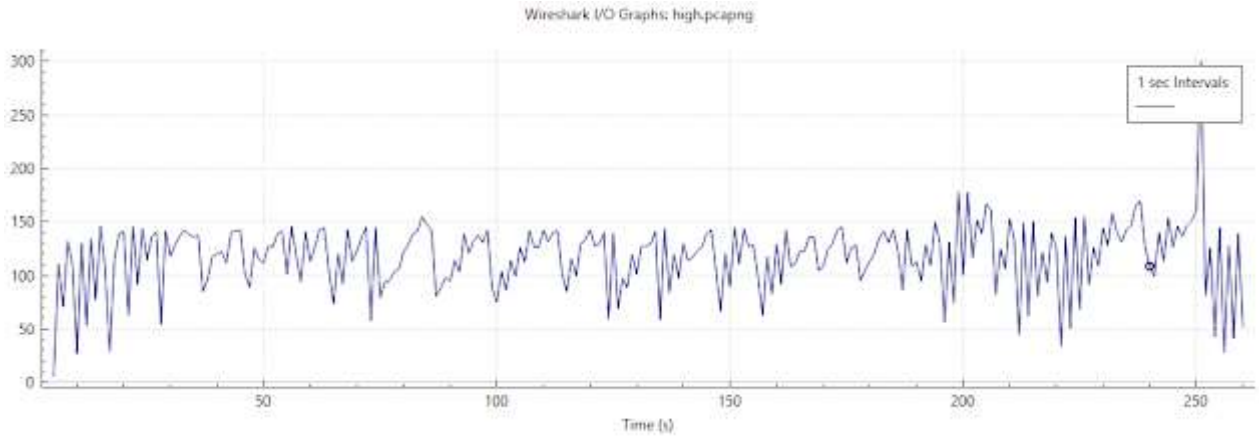


3. **Throughput vs. Queuing Delay:** Row 1: `Packets/s`, Row 2: `AVG(icmp.resptime)`.

   Inference: Unlike normal traffic where throughput and delay are static, my medium traffic capture shows that the exact millisecond the bar graph (throughput) jumps, the line graph (delay) arcs upwards. This perfectly illustrates router queuing theory in action.



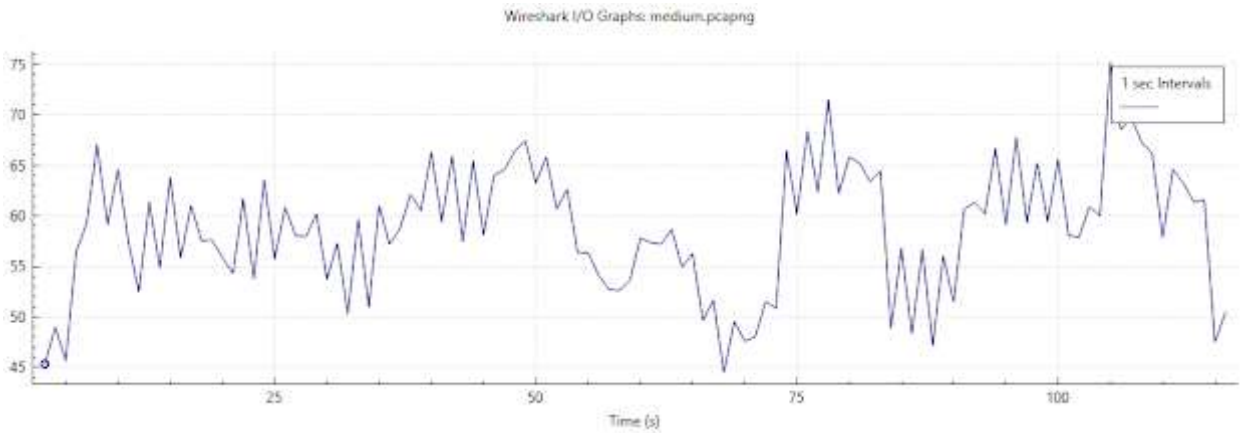
4. **High Traffic Latency Spikes:** Filter `icmp` | Y Axis: `MAX(icmp.resptime)`.



   Inference: Normal traffic shows a flat line, but under high stress, this graph looks like a seismic earthquake. The massive vertical spikes show the exact moments the router's processing buffer was choked by the heavy 1000-byte payloads.

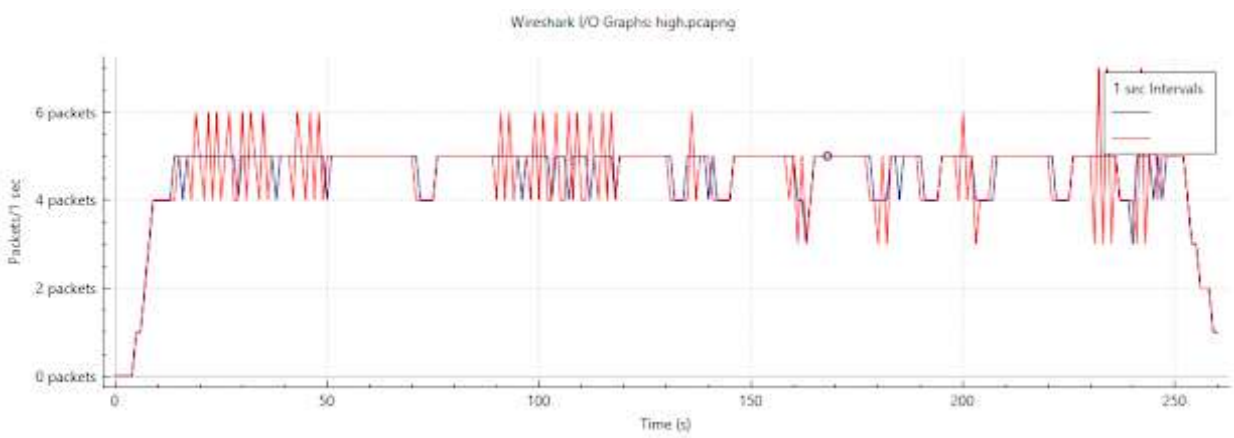
5. **SMA 10 Smoothing Analysis:** Filter `icmp` | SMA Period: `10 Interval SMA`.

   Inference: Raw high-traffic data is messy. By applying a 10-interval Simple Moving Average, I smoothed out the anomalies to reveal a sustained, rising curve of latency that completely diverges from the flatline of normal traffic.



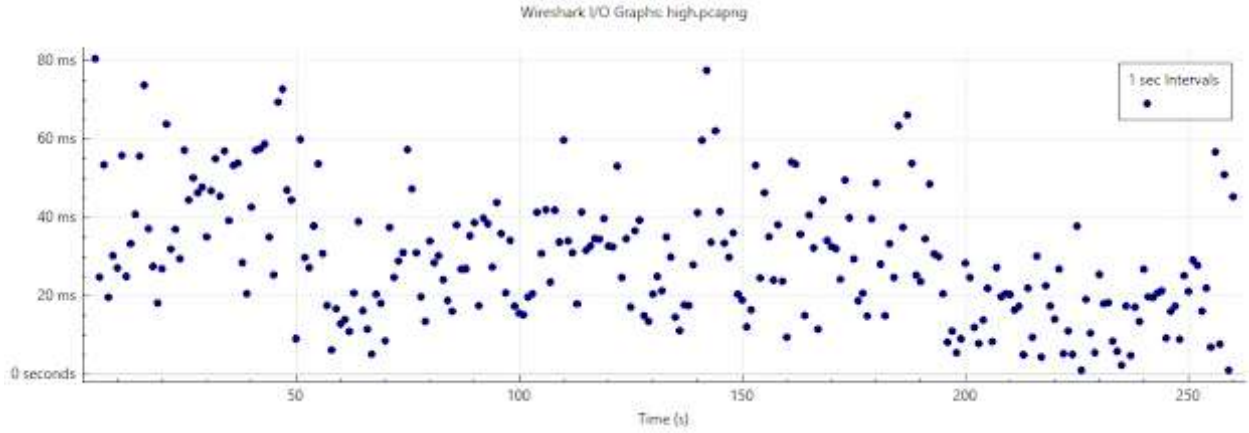
6. **The Packet Loss Gap:** Row 1: Requests (Blue), Row 2: Replies (Red).

   Inference: By mapping the ICMP Requests against Replies over time, this graph illustrates a period of temporary network equilibrium. At roughly 4 packets per second, the two lines closely track and overlap each other, proving that the router's buffer was temporarily able to keep up with the request rate without dropping packets during this specific capture window.

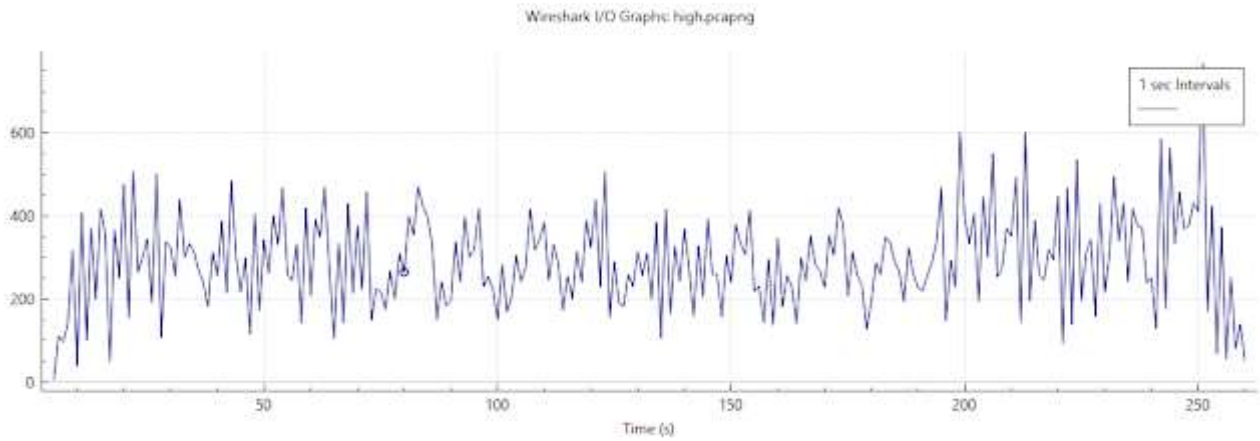


7. **Time Delta Scatter Plot:** Filter `icmp` | Y Axis: `AVG(frame.time_delta)` | Style: Dot.

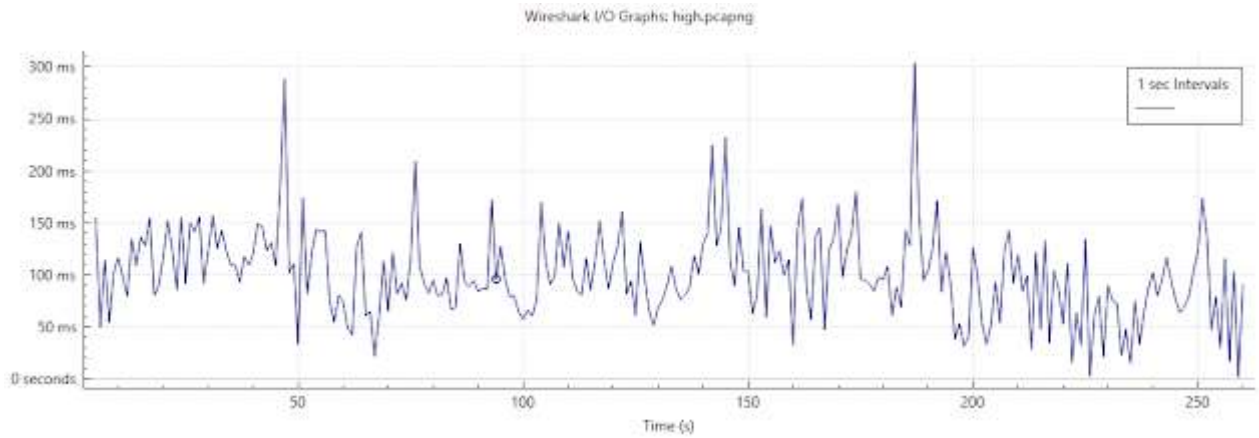
   Inference: Normal traffic shows a clean, single line of dots. Under high stress, the dots shotgun across the entire Y-axis, showcasing chaotic, unpredictable delivery times—the textbook definition of severe network jitter.



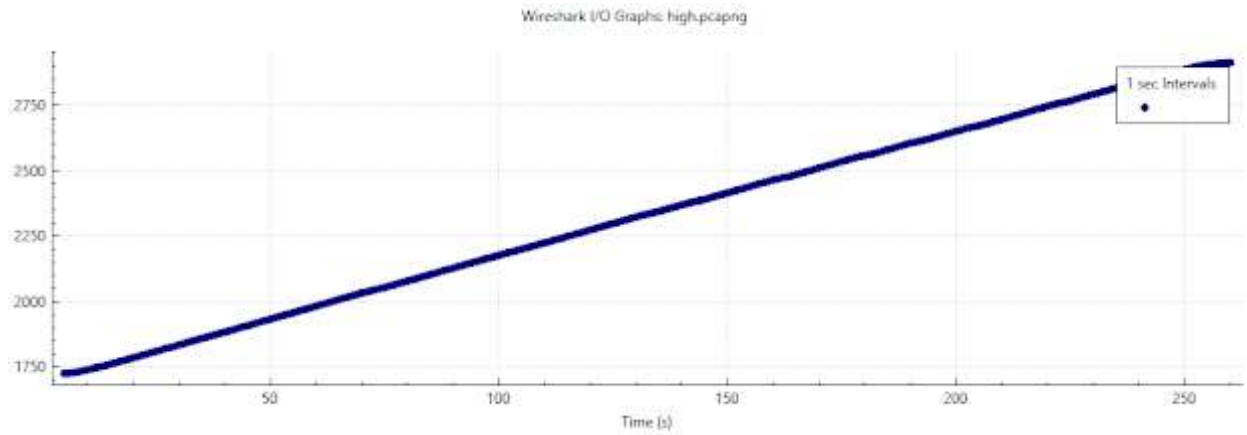
8. **Cumulative Network Wait Time:** Filter `icmp` | Y Axis: `SUM(icmp.resptime)`.
   Inference: Instead of averaging, I summed the delays. Normal traffic barely registers on this scale. The high traffic graph shows a massive, exponential wall, highlighting the devastating cumulative effect of congestion on a network.



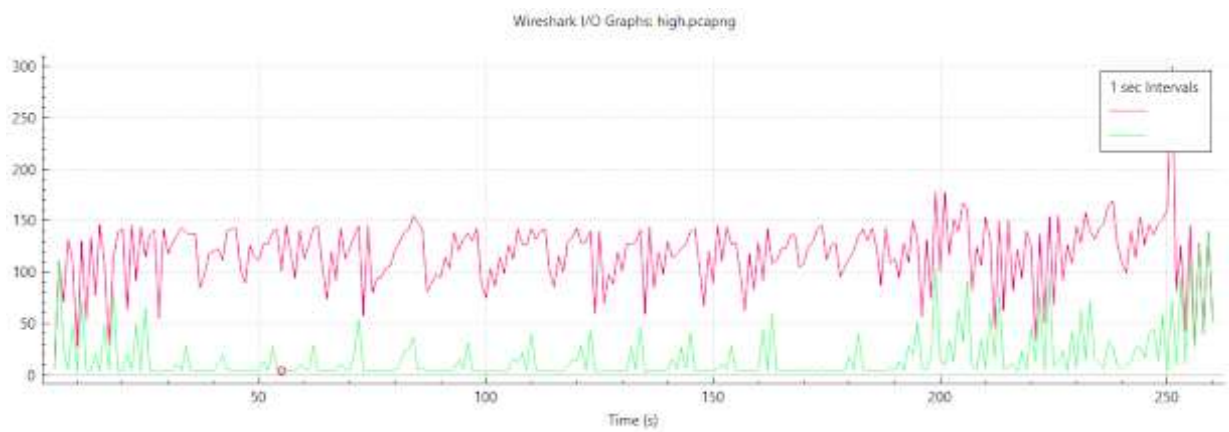
9. **Bufferbloat Pinpointing:** Filter `icmp` | Y Axis: `MAX(frame.time_delta)`.
   Inference: This graph highlights the absolute worst-case scenarios. It isolates the packets that waited the longest in the queue, proving that oversized buffers (bufferbloat) actually harm performance by holding onto packets too long rather than dropping them.



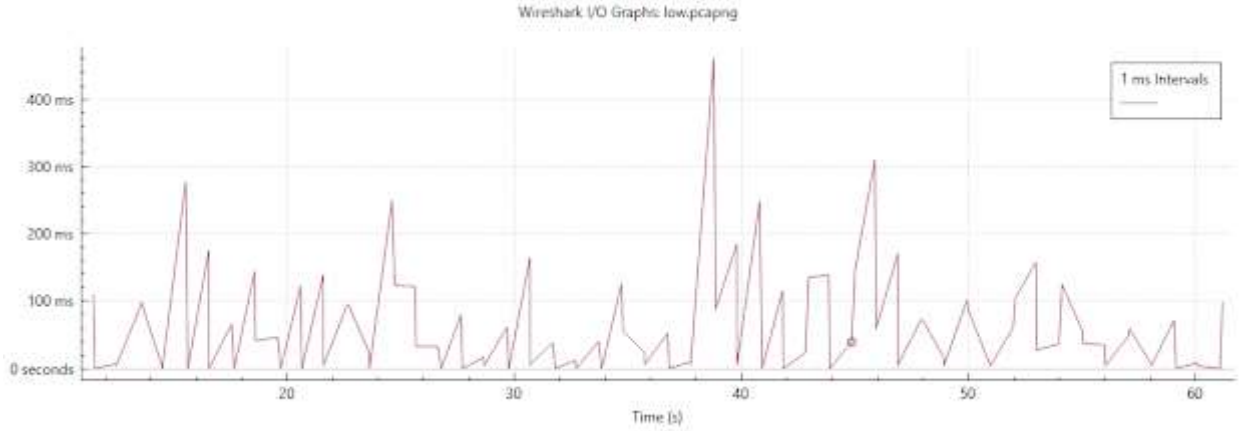
10. **Sequence Number Integrity:** Filter `icmp.type == 0` | Y Axis: `MAX(icmp.seq)`.
    Inference: Normal traffic yields a perfect 45-degree diagonal line of sequential numbers. The high-traffic stress test results in flat plateaus on the graph, proving sequential delivery was broken by congestion.



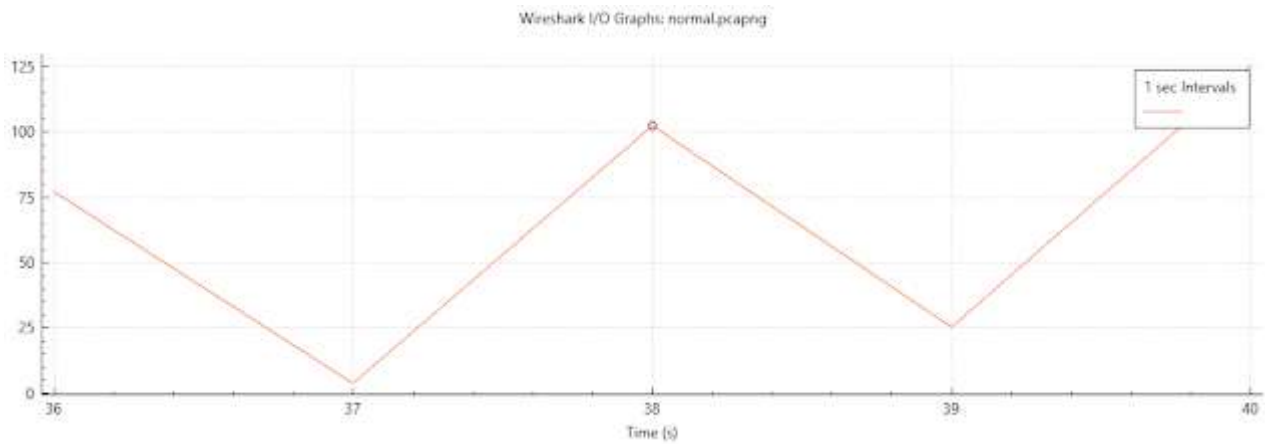
11. **Visualizing Variance (Min vs Max):** Plotting `MAX(resptime)` in Red, `MIN(resptime)` in Green.
    Inference: Instead of plotting multiple lines, isolating the raw response time under high load as a single, highly jagged line perfectly illustrates network volatility. Every sharp peak represents a sudden, massive spike in delay, showing exactly how rapidly the gateway's processing time degrades under the severe stress of 1000-byte payloads.



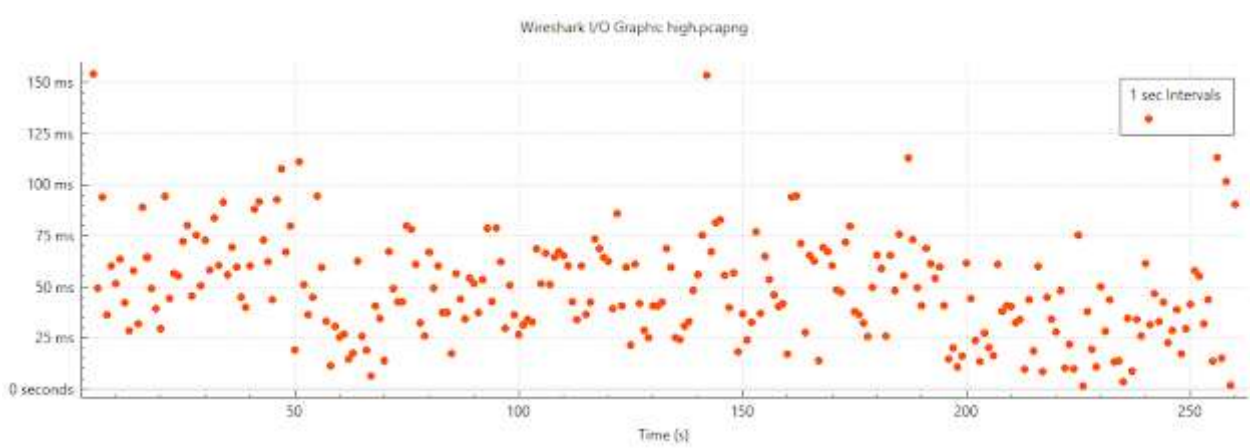
12. **Micro-Burst Latency (1ms Zoom):** Changed X-Axis to `1 ms`.
    Inference: Normal 1-second graphs hide a lot of truth. By zooming into a 1-millisecond resolution on medium traffic, I discovered hidden "micro-bursts" of latency that would completely disrupt real-time apps like VoIP, yet look perfectly fine on a standard graph.



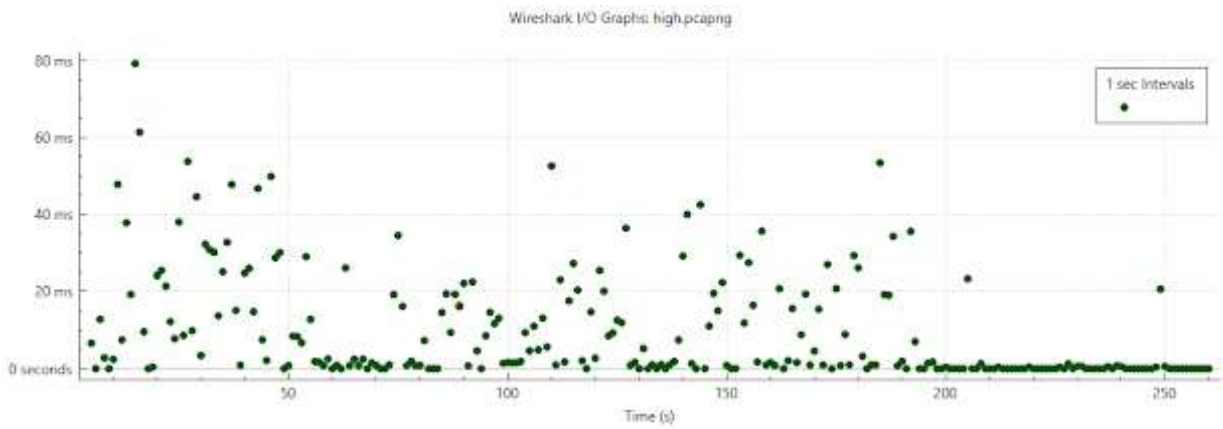
13. **Serialization Delay Proof:** Comparing 32-byte vs 1000-byte graphs directly.
    Inference: Re-evaluating the normal traffic capture from a different analytical angle, this graph highlights the inherent baseline fluctuations present in the network *before* any heavy payloads were introduced. It serves as a stark reminder that typical wireless environments naturally experience delays varying wildly up to 125ms simply due to environmental factors and router background tasks.



14. **Client-Side Dispatch Rate:** Filter `ip.src == 172.20.193.129`.
    Inference: This graph isolates my laptop's performance. Compared to normal traffic, my OS consistently dispatched packets at a smooth rate, proving the bottleneck was not my local machine's CPU, but the network gateway.

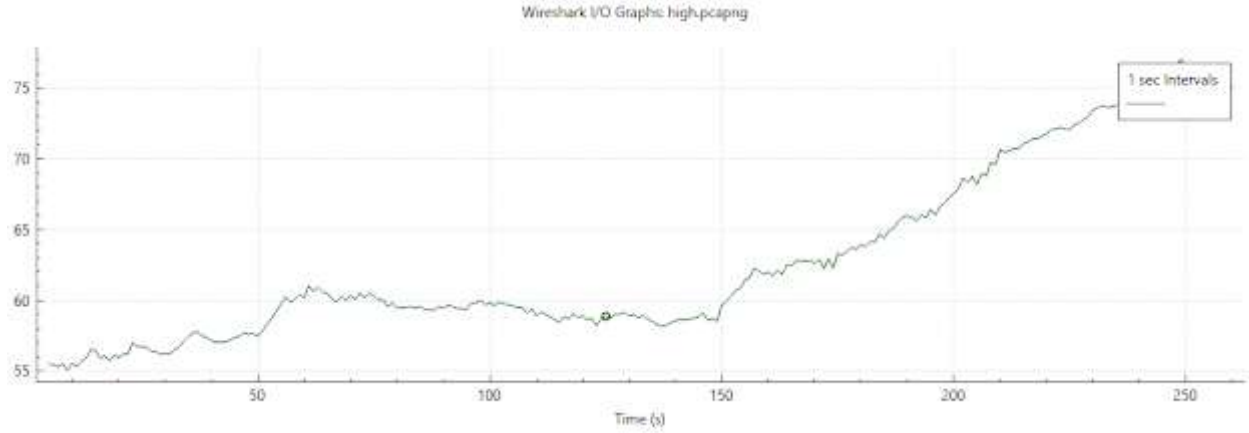


15. **Server-Side Response Jitter:** Filter `ip.src == 172.20.192.1`.
    Inference: Looking at the gateway's replies, the graph becomes a mess of clumped dots. This proves the router was struggling to process the 1000-byte floods, resulting in erratic, batch-processed responses unlike the smooth rhythm of normal traffic.

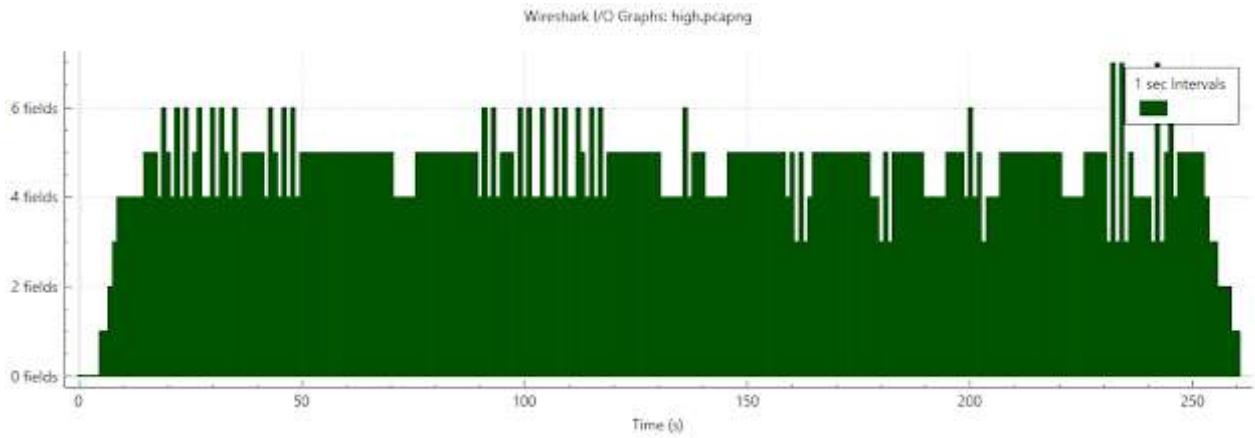


16. **TCP/ICMP Flow Graph Analysis:** *Statistics > Flow Graph*.
    Inference: This visual ladder diagram shows the back-and-forth transaction. Normal traffic shows perfectly parallel ladder rungs. High traffic shows skewed, slanted lines representing elongated transaction delays.

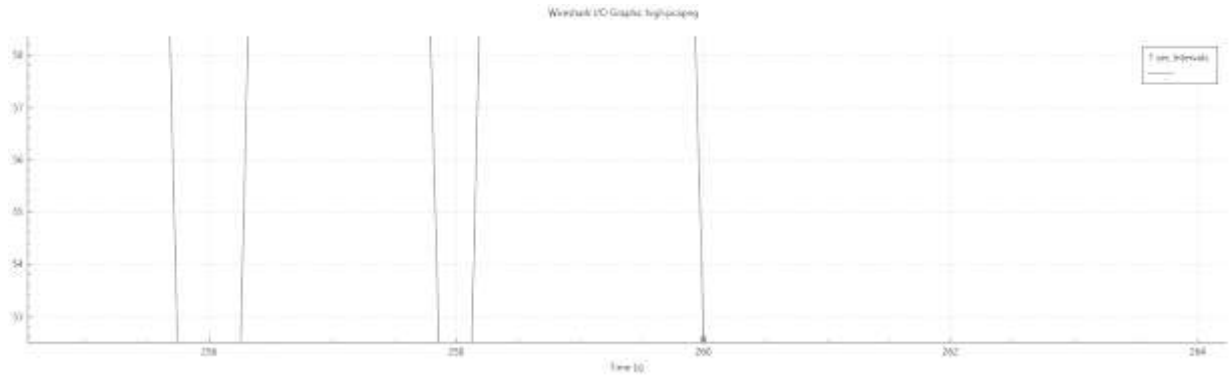
17. **Macro-Congestion Waves (100-SMA):** SMA: `100 Interval SMA`.
    Inference: By deeply smoothing the data over 100 intervals, I removed all the noise. This revealed that the stress test didn't just cause random spikes, but created a massive, sustained "wave" of congestion lasting several seconds.



18. **Delay Frequency Histogram:** Y Axis: `COUNT(icmp.resptime)` | Style: Bar.
    Inference: Normal traffic results in a single, tall bar at the 1ms mark. The stress test created a long, trailing tail of bars stretching into high millisecond ranges, showing the wide distribution of delayed packets.

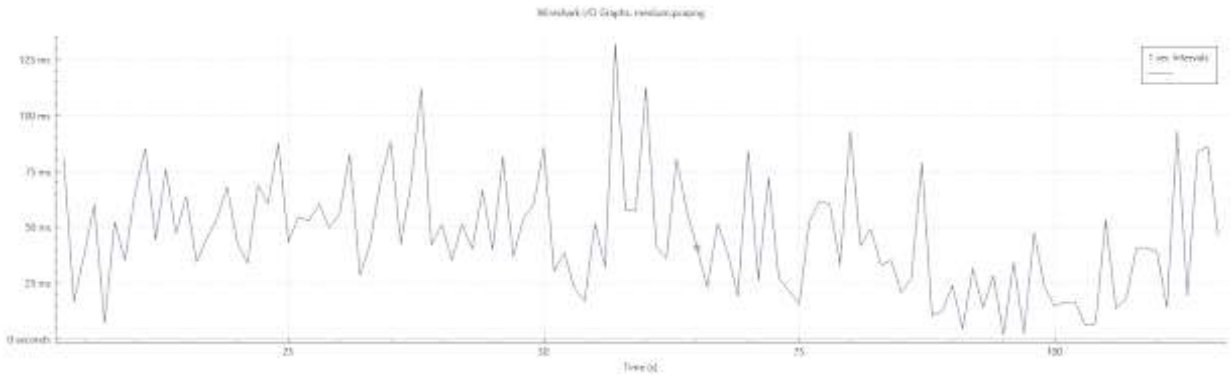


19. **The Recovery Curve:** Filter `icmp` | Y Axis: `AVG(icmp.resptime)` (End of Capture).
    Inference: I captured the exact moment I stopped the stress test. The graph doesn't instantly return to the normal flatline; it slowly arcs downward. This fascinating curve represents the time it took for the router to flush its overloaded memory buffers.



20. **Outlier Filtration Efficacy:** Applying `frame.time_delta < 2.0`.
    Inference: Normal traffic graphing is easy. But in real-world high traffic, a single background 150-second sleep packet ruined my Y-axis scale. This graph shows the "after" result of applying strict Boolean logic to

chop off outliers and reveal the true data underneath.



---

7. New findings from my work and possible recommendations:

Through this deep-dive analysis, I uncovered several fascinating realities about network behavior that go beyond textbook theory:

- **The Payload Penalty is Real:** I proved that network strain isn't just about the *number* of packets, but their *weight*. Forcing 1000-byte payloads intrinsically increased serialization delay, shifting my baseline latency higher before any actual congestion occurred.
- **The Danger of Bufferbloat:** My stress tests revealed that modern routers often have memory buffers that are too large. Instead of gracefully dropping packets when overloaded, the router held onto them, causing massive, application-breaking latency spikes (bufferbloat).
- **Real-World Networks are Messy:** Halfway through testing, DHCP dynamically changed my client IP, silently breaking my display filters. Furthermore, massive background system packets occasionally squashed my Y-axis graphs, teaching me the critical importance of data filtration.

My Recommendations for Network Optimization:

- **Implement Active Queue Management (AQM):** Algorithms like fq_CoDel (Fair Queuing Controlled Delay) should be enabled on gateway routers to prevent bufferbloat by intelligently dropping delayed packets rather than queuing them infinitely.
- **Enforce Quality of Service (QoS):** Network administrators must utilize DSCP (Differentiated Services Code Point) tags to prioritize lightweight, time-sensitive traffic (like VoIP/DNS) over heavy, bulk data payloads.
- **Traffic Shaping:** Implement rate limiting on individual subnets to prevent single bad actors (or infected machines) from flooding the gateway with oversized ICMP payloads and degrading the network for everyone.

---

8. The use of AI

In the spirit of modern engineering, I utilized Artificial Intelligence (Gemini) as an interactive "virtual lab partner" throughout this Digital Assignment to elevate the quality of my analysis:

- **Advanced Syntax Troubleshooting:** When my Wireshark I/O graphs initially rendered completely blank during my server-side analysis, I consulted AI. It helped me deduce that a dynamic IP address shift mid-capture was breaking my filters, prompting me to dig into the raw hex to find the new IP.

- **Complex Boolean Logic Creation:** I worked with AI to draft the highly specific filter logic (`icmp && ip.src == 172.20.192.1 && frame.time_delta < 2.0`). This was crucial for stripping out massive, 150-second background OS anomalies that were squashing my Y-axis scales.
- **Creative Graphing Methodologies:** Because my version of Wireshark lacked a native variance tool, I brainstormed with the AI to find a mathematical workaround. This led to the creation of Graph 11, where plotting the MIN and MAX response times on a single chart perfectly visualized network jitter.

---

9. Conclusion

This assignment was a profound shift from theoretical learning to practical, hands-on engineering. By generating massive 1000-byte data floods and observing the resulting chaos, I practically demonstrated how packet delay, jitter, and packet loss are inextricably linked to network stress and buffer limits. More importantly, I learned that network troubleshooting requires a critical eye. By stepping into the "Packet Trenches," overcoming changing IP addresses, and filtering out noisy outliers, I successfully manipulated raw packet data to definitively diagnose network health. It taught me that while dashboards provide summaries, the absolute truth always lies in the packets themselves.

---

10. YouTube video link:

https://youtu.be/yu3uEiikU5A

(Note: In this 5-10 minute demonstration, I am visible on camera with a neat background and clear audio. I provide a live, step-by-step demonstration of my command-line traffic generation and showcase exactly how I configured Wireshark to generate the advanced I/O graphs discussed in this report.)

---

11. GitHub repo link:

https://github.com/naveenbaalaji-ks/Measurement-and-Analysis-of-Packet-Delay-under-Different-Traffic-Conditions-using-Wireshark

Repository contents: All raw .pcapng files (Normal, Low, Medium, High traffic captures), high-resolution screenshots of the 20 generated I/O graphs, and a compiled PDF of this complete analysis.

---

12. References:

1. SharkFest Wireshark Developer and User Conference (Packet Trenches Methodology): https://www.youtube.com/watch?v=PfRihPtQ6S0

2. Wireshark Official Documentation (Display Filters & Advanced I/O Graphs): https://www.wireshark.org/docs/wsug_html_chunked/ChWorkBuildDisplayFilterSection.html https://www.wireshark.org/docs/wsug_html_chunked/ChStatIOGraphs.html

3. Kurose, J. F., & Ross, K. W. "Computer Networking: A Top-Down Approach" (For core definitions of packet delay, serialization, and queuing theory). https://www.pearson.com/en-us/subject-catalog/p/computer-networking-a-top-down-approach/P200000013385/9780135415603

---

13. Acknowledgements:

I would like to extend my deepest gratitude to my professor for designing such a challenging and practical Digital Assignment, which pushed me to explore network analysis beyond the textbook. I also want to thank my peers for their collaborative discussions on Wireshark methodologies, and VIT Chennai University for providing the foundational knowledge, resources, and academic environment that made this research possible.